Data Structures (ETCCDS202)

Assignment - 4

Name of the School: School of Engineering & Technology

Program: B. Tech CSE (AI & ML)

Course Title: Data Structures

Course Code: ETCCDS202

Unit Title: Non-Linear & Advanced Data Structures

Student Name: Abhay Kumar

Roll No.: 2501730185

Section: A

1. Introduction

Non-linear data structures play a critical role in modern computing systems where hierarchical and network-based relationships are required. Unlike linear structures, these structures enable efficient **searching, indexing, routing, and data organization**.

This assignment focuses on implementing and understanding three essential structures:

- **Binary Search Tree (BST)** – for ordered data storage
- **Graph (Adjacency List with BFS & DFS)** – for network traversal
- **Hash Table (Separate Chaining)** – for fast key-value access

Together, these demonstrate core computational strategies used in **databases, operating systems, caching, and networking applications**.

2. Binary Search Tree (BST)

2.1 Concept and Properties

A **Binary Search Tree** is a hierarchical structure with the following defining property:

- All nodes in the **left subtree** are smaller than the root
- All nodes in the **right subtree** are greater than the root

This property enables efficient searching and maintains **sorted order implicitly**.

2.2 Operations Implemented

Insertion

- A new node is inserted recursively at the correct position.
- Maintains BST ordering property.

Search

- Starts from the root and compares values.
- Traverses left or right depending on the key.

Deletion

Deletion is the most critical BST operation and includes three cases:

1. **Leaf Node (No Child)**
   - Node is simply removed.
2. **Node with One Child**
   - Node is replaced by its child.
3. **Node with Two Children**
   - Node is replaced with its **inorder successor** (minimum value in right subtree).

2.3 Inorder Traversal and Sorted Output

Inorder traversal follows:

Left → Root → Right

Because of BST ordering:

- Left subtree contains smaller values
- Root is intermediate
- Right subtree contains larger values

Thus, inorder traversal always produces **sorted (ascending) output** without requiring additional sorting algorithms.

2.4 Time Complexity Analysis

Operation	Average Case	Worst Case
Insert	O(log n)	O(n)
Search	O(log n)	O(n)
Delete	O(log n)	O(n)

- **Average Case:** Balanced tree
- **Worst Case:** Skewed tree (degenerates into linked list)

2.5 Practical Applications

- Database indexing (e.g., search trees)
- File system organization
- Ordered data retrieval systems

3. Graph Representation and Traversals

3.1 Concept

A **graph** is a non-linear data structure consisting of:

- **Vertices (nodes)**
- **Edges (connections)**

In this assignment, the graph is:

- **Directed** (edges have direction)
- **Weighted** (edges carry cost/weight)
- Stored using an **adjacency list**

3.2 Adjacency List Representation

- Each vertex maintains a list of its connected neighbors.
- Each connection stores both **destination and weight**.
- Efficient in terms of space for sparse graphs.

3.3 Breadth-First Search (BFS)

Concept

- Traverses graph **level by level**
- Uses a **queue (FIFO)**

Working Principle

1. Start from a given node
2. Visit all adjacent nodes
3. Move to the next level

Key Feature

- Guarantees shortest path in **unweighted graphs**

Applications

- Network routing
- Social network analysis
- Shortest path problems

3.4 Depth-First Search (DFS)

Concept

- Traverses graph by going **as deep as possible first**
- Uses **recursion or stack**

Working Principle

1. Start from a node
2. Explore one branch completely
3. Backtrack and explore remaining branches

Applications

- Cycle detection
- Topological sorting
- Path finding

3.5 BFS vs DFS (Comparison)

Feature	BFS	DFS
Traversal	Level-wise	Depth-wise
Data Structure	Queue	Stack / Recursion
Optimal For	Shortest path	Deep exploration
Memory Usage	Higher	Lower (generally)

4. Hash Table with Separate Chaining

4.1 Concept

A **hash table** is a data structure that maps keys to values using a **hash function**, enabling near constant-time operations.

4.2 Hash Function Used

A simple modulo-based hash function:

index = key % table_size

This converts a key into an index of the table.

4.3 Collision and Its Cause

A **collision** occurs when:

- Two or more keys map to the same index

Example:

- 10 % 5 = 0
- 15 % 5 = 0

Both keys collide at index 0.

4.4 Separate Chaining Technique

- Each index stores a **list (chain)** of key-value pairs
- Colliding elements are stored in the same list

Advantages

- Prevents data loss due to collisions
- Maintains simplicity and flexibility

4.5 Operations Implemented

Insert

- Adds element to the appropriate chain

Search (Get)

- Traverses chain to find key

Delete

- Removes specific key from chain

4.6 Time Complexity

Operation	Average Case	Worst Case
Insert	O(1)	O(n)
Search	O(1)	O(n)
Delete	O(1)	O(n)

- Worst case occurs when all keys fall into the same bucket

4.7 Applications

- Hash maps and dictionaries
- Caching systems (e.g., web caching)
- Database indexing
- Symbol tables in compilers

5. Observations

- BST performance depends heavily on tree balance

- BFS ensures optimal traversal for shortest paths, while DFS is efficient for deep analysis
- Hash tables provide the fastest access but depend on a good hash function and collision handling

Each structure is optimized for a different problem type, highlighting the importance of choosing the right data structure.

6. Conclusion

This assignment demonstrates the practical implementation and importance of **non-linear and advanced data structures**:

- **BST** enables efficient ordered storage and retrieval
- **Graphs** model real-world systems such as networks and routing
- **Hash Tables** provide near constant-time data access

Understanding these structures is essential for designing **efficient algorithms and scalable systems** in real-world applications.